

① Comparative Analysis

Compare hashing and binary search tree based on time complexity and methods.

② Compare the time complexity of all notations in bubble, insertion, selection, merge, quick and heap.

Ans:

Feature	Hashing	Binary Search tree
search time	$O(1)$ (constant average time)	$O(\log n)$ (logarithmic time)
Insertion/ deletion	$O(1)$ (average)	$O(\log n)$ (guaranteed)
Worst case performance	$O(n)$ (if many collisions occur.)	$O(\log n)$ (for AVL / Red-Black tree)
data ordering	unordered (data is randomized)	Ordered (naturally sorted)

Range
Querymemory
allocationCore
mechanism②
StackSpace
complexityWorst
CaseAverage
Case

Best Case

25/11/24
based

Range Query	Inefficient (requires checking every time)	Highly efficient ($O(k + \log n)$ to find a range)
memory allocation	Allocates contiguous chunks upfront	Dynamic; allocates memory per node
Core mechanism.	Hash function and array index	node link based on value comparison

Sorting Algorithm	Best case	Average case	Worst case	Space complexity	Stable
Bubble sort	$O(n^2)$ (with flag)	$O(n^2)$	$O(n^2)$	$O(1)$ (inplace)	yes
Insertion sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$ in place	yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$	no
merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$ yes	
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ no stack space	
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ (inplace)	no

$\frac{2n}{2}$